

Week 06

Assertion-Based Verification

Jesus Sanchez

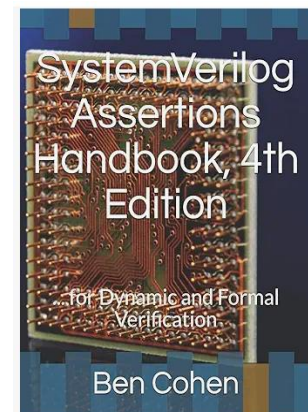
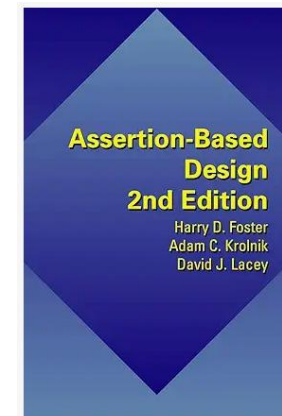
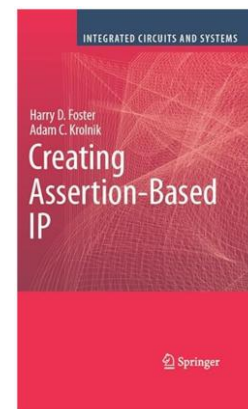
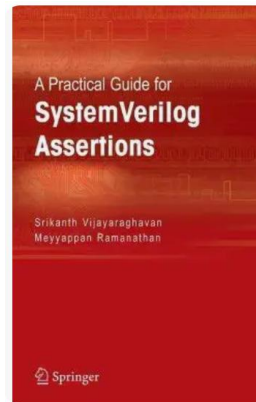
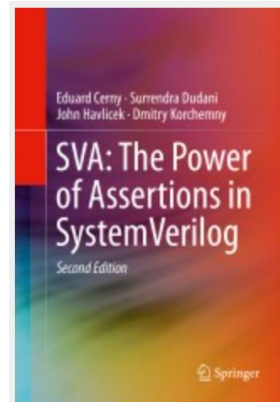
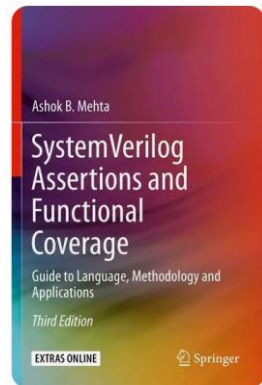
Dept. d'Arquitectura de Computadors
Universitat Politècnica de Catalunya

Processor Verification

Master in Innovation and Research in Informatics

Objectives of the Week

- Understand the role of assertions in simulation and formal verification
- Differentiate between **immediate** and **concurrent** assertions and their use cases



Agenda

1. Introduction to assertions
2. Immediate assertions
3. Concurrent assertions
4. Advanced concurrent assertions
5. A tricky example
6. Wrap-up and self-learning

What Are Assertions?

- **Assertions** are statements that check whether a specific condition holds
- **Why use assertions?**

Automatic Checks

- Assertions **monitor** design behavior
- **Immediate** alerts for unexpected events

Better Debugging

- Pinpoints exactly **where and why** a failure occurs

Improved Quality

- Encourages **specification clarity**
- Assertions are part of the **design documentation**

Formal Verification

- Assertions are used by **formal tools** to exhaustively check design correctness

Assertions act as **automatic checkers** to ensure your design behaves as expected

Types of Assertions

Immediate Assertions

- Executed **immediately** in simulation flow
- Must reside in procedural code (initial, always blocks)
- Syntax:
`assert (expression) else ...`

Instant checks

Concurrent Assertions

- Evaluate **over simulation time** (clock cycles, events)
- Useful for protocols, handshake signals, sequences
- Syntax:
`assert property (property_expression)`

Time-based checks

Where to Place Assertions in Your Testbench?

- Inside the DUT (Design Under Test)

- In RTL modules, assertions ensure design-level correctness

- In the Testbench

- Assertions validate stimulus generation and expected DUT behavior

- In Monitors and Scoreboards

- Embedded in verification components to ensure protocol and interface compliance

- In Interfaces and Transactions

- Assertions tied to communication and data transfers

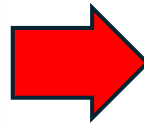
Who is Responsible for Adding Asserts in the DUT?

- Typically, **designers** are responsible for adding immediate assertions in the RTL code, because they have the best knowledge of what's “correct” at the design level
- Can the verification engineer add assertions in the DUT?
 - Not usually:
 - In many organizations, the verification engineer cannot touch the RTL
 - Modifying RTL for verification may also invalidate integrity and synthesis results
 - Instead, they insert assertions around the DUT in testbenches, interfaces, or monitors
- If we cannot modify the DUT?
 - If adding assertions to the RTL is not possible, verification engineers can still:
 - Add **interface-level assertions** using SystemVerilog interfaces.
 - Use **bind statements** to “bind” assertions to signals without changing RTL code.

bind Example

“When **req** is asserted,
ack must eventually be asserted within 2 cycles”

```
module my_dut (  
    input logic clk,  
    input logic reset,  
    input logic req,  
    output logic ack  
);  
    always_ff @(posedge clk or posedge reset)  
        if (reset)  
            ack <= 0;  
        else if (req)  
            ack <= 1;  
  
    property ack_follows_req;  
        @(posedge clk) disable iff (reset)  
        req |-> ##1 ack;  
    endproperty  
  
    a1: assert property (ack_follows_req);  
endmodule
```



```
module assertions(  
    input logic clk,  
    input logic reset,  
    input logic req,  
    input logic ack  
);  
    property ack_follows_req;  
        @(posedge clk) disable iff (reset)  
        req |-> ##1 ack;  
    endproperty  
  
    a1: assert property (ack_follows_req);  
endmodule
```

```
module my_tb;  
    logic clk, reset, req, ack;  
  
    my_dut u_dut (.clk(clk), .reset(reset), .req(req), .ack(ack));  
  
    bind my_dut assertions my_asserts (clk, reset, req, ack);  
endmodule
```


Agenda

1. Introduction to assertions
2. Immediate assertions
3. Concurrent assertions
4. Advanced concurrent assertions
5. A tricky example
6. Wrap-up and self-learning

What Are Immediate Assertions?

■ When

- Executed in the **current simulation time** (same as the simulation statement)
- Checked **immediately** during simulation (not over multiple cycles)

■ Placement

- Immediate assertions can be used inside procedural blocks:
initial, always, always_ff, always_comb, always_latch

■ Usage

- Catch unexpected behavior instantly $\Rightarrow$ No temporal delay

Immediate assertions **cannot describe temporal** (multi-cycle) behavior,
that's what concurrent assertions are for!

Basic Syntax and Examples

- Syntax

```
assert (<boolean_expression>) [action_if_true]  
[else action_if_false];
```

- Examples

```
always_ff @(posedge clk) begin  
    // Condition: ack can be active only when there is a request  
    assert (ack == 0 || req == 1) $display("ack is correct");  
    else $error("ack is high without req!");  
end
```

```
always_ff @(posedge clk) begin  
    // Condition: if a transaction is active, data must be valid and not zero  
    assert (!(transaction_active) || (data_valid && data != 0))  
    else $fatal(1, "Transaction active but data is invalid/zero at time %t", $time);  
end
```

Handling Assertion Failures

■ What Happens When an Assertion Fails?

- The simulation prints a **warning** or **error message** (by default).
- Actions can be customized using system tasks:

\$error	Logs an error but allows simulation to continue
\$warning	Logs a warning but does not affect simulation control
\$fatal	Halts simulation immediately
\$finish	Ends simulation gracefully at the end of the test

■ Best Practices

- Use **\$fatal** for critical design errors
- Use **\$error** for design **bugs that require investigation** but not a full stop

Customize messages to include context (e.g., \$time, instance name).

Agenda

1. Introduction to assertions
2. Immediate assertions
3. Concurrent assertions
4. Advanced coverage features
5. A tricky example
6. Wrap-up and self-learning

What Are Concurrent Assertions?

- Assertions that monitor temporal relationships between signals and events
 - Evaluate over multiple simulation cycles, unlike immediate assertions
- Why Use Concurrent Assertions?
 - Ideal for checking sequential behaviors:
 - e.g., “If signal A goes high, then B must go high within 3 cycles.”
- How Are They Specified?
 - Typically expressed as a SystemVerilog property (using property and sequence blocks)
 - Instantiated using assert property
- Key Features
 - Express sequential constraints
 - Work outside procedural blocks (but can be bound to them using bind)

Basic Syntax

■ Syntax

```
sequence <sequence_name> [arguments];  
    <sequence_expression>;  
endsequence
```

```
property <property_name> [arguments];  
    <sequence_or_property_expression>;  
endproperty
```

```
assert property (<property_name>) [action_if_true;]  
    [else action_if_false];
```

Promotes reuse, clarity, and scalability in complex verification projects

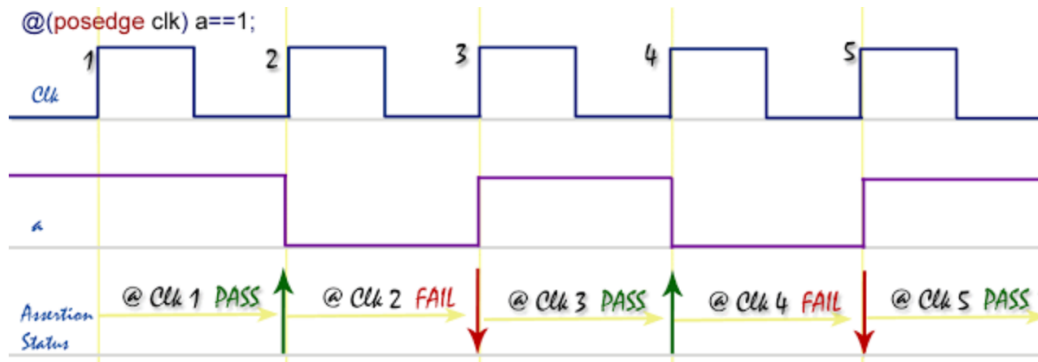
Sequences: The Building Blocks

■ What Are Sequences?

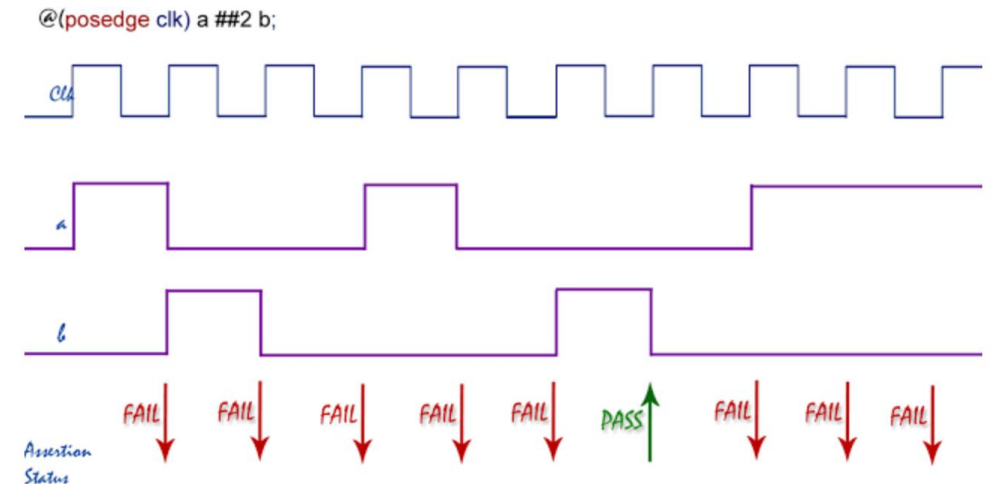
- Describe **ordered behavior** over time, involving **single or multiple** clock cycles
- Think of them as a set of temporal conditions

■ Why Use Sequences?

- They define the timing of events (e.g., “req should be followed by ack within 3 cycles”)
- Reusability: can be referenced by multiple properties or assertions



Source: [SystemVerilog assertion Sequence - Verification Guide](#)



Properties: Combining Sequences

■ What Are Properties?

- A property describes a **complete temporal relationship** that can be checked

■ Using Sequences in Properties

- You can reference sequences directly or combine them with operators
- You can combine sequences to describe more complex timing relations

```
property full_transaction;  
    start_seq |-> response_seq;  
endproperty
```

Sequences : Define temporal patterns of signals (like building blocks)

Properties : Use sequences to assert conditions (like rules or contracts)

Simple Concurrent Assertion Examples

```
// Grant should occur 1 cycle after request
sequence s_grant_after_req;
    req ##1 grant;
endsequence

property p_valid_grant;
    @(posedge clk)
    disable iff (reset)
    s_grant_after_req;
endproperty

assert property (p_valid_grant);
```

```
// After reset deasserts signal must be 0 for 3 cycles
property reset_behavior;
    @(posedge clk) !reset |-> (signal == 0)[*3];
endproperty

assert property (reset_behavior);
```

Temporal Operators

Sequence →

Property →

Operator	Name	Description
##	Delay operator	Specifies delay between events or conditions in cycles
->	Overlapping implication	Occurs in current cycle
=>	Non-overlapping implication	Occurs after current cycle

```
sequence s1;  
  a ##2 b; // b must occur 2 cycles after a  
endsequence
```

```
// If 'a' is true, 'b' must also be true at the same time  
property p1;  
  @(posedge clk) a | -> b; ← a and b can be sequences!  
endproperty
```

```
// If 'a' is true at time t, then 'b' must be true starting at t+1  
property p2;  
  @(posedge clk) a | => b; ←  
endproperty
```

Implication Operators

■ Overlapping Implication ($| \rightarrow$)

- Checks if consequent overlaps with antecedent or begins immediately
- Used when both conditions may happen in the same clock cycle

```
// If a at t, then b at t and c at t+1  
a |→ b ##1 c;
```

■ Non-Overlapping Implication ($| \Rightarrow$)

- Checks if consequent starts after the antecedent completes
- Safer for modeling sequential behavior

```
// If a at t, then b at t+1 and c at t+2  
a |⇒ b ##1 c;
```

disable iff for Conditional Assertions

- Purpose of disable iff

- **disable iff** is used to conditionally disable a concurrent assertion when a specified condition is true
- Common use case: reset conditions — we don't want assertions to fire during reset or initialization

- Syntax Overview

```
    assert property (@(posedge clk)
        disable iff (reset)
        sequence_expression
    );
```

- **disable iff** (reset) disables the assertion while reset is true
- The sequence is not evaluated in this case

- Why Use It Instead of if?

- **disable iff** removes the assertion entirely from evaluation
- An **if (reset) ... else ...** still evaluates the expression
- **disable iff** avoids false positives during known invalid states

- Best Practices

- Always use **disable iff** to mask assertions during: reset, init states, illegal DUT modes

- Example: Assertion With Reset

```
property p_valid_stable;
    @(posedge clk)
    disable iff (reset)
    valid |-> ##1 stable;
endproperty
assert property (p_valid_stable);
```

- If reset == 1, this assertion is disabled for that cycle
- When reset == 0, the assertion becomes active

10:00

Agenda

1. Introduction to assertions
2. Immediate assertions
3. Concurrent assertions
4. Advanced concurrent assertions
5. A tricky example
6. Wrap-up and self-learning

Repetition and Range Operators

- Why Use Repetition in Assertions?
Repetition operators can be combined with sequences
 - Many behaviors in hardware are repetitive or time-variable
 - SystemVerilog provides temporal repetition operators to express:
 - Fixed-length/variable-length repetition and/or One-or-more/zero-or-more behaviors

Operator	Meaning	Example
Consecutive Repetition		
[*N]	Repeat exactly N consecutive cycles	A [*3] → A, A, A
[*N:M]	Repeat between N and M cycles (inclusive)	A [*2:5]
[*]	Repeat zero or more cycles	A [*]
Non-Consecutive Repetition		
[->N]	Match N occurrences (non-consecutive)	A [->2] → A occurs twice, not necessarily in a row
[->N:M]	Match between N to M occurrences	A [->2:4]
[->]	Match any number of occurrences	A [->]

Multicycle Path Checking

■ What Is Multicycle Path Checking?

- Used to verify conditions that must hold across multiple clock cycles, especially for:

- Latency-sensitive operations
- Multi-stage pipelines
- Bus transactions and protocols
- Longer reaction windows

Operator	Meaning
##N	Delay of exactly N clock cycles
##[N:M]	Delay between N and M cycles

- In SystemVerilog, this is typically done using **sequences** and **temporal delays**.

■ Examples of use:

- A response follows a request **within N cycles**
- A signal remains stable for **several cycles**
- A state transition **completes over a range of time**

Quick Quiz

EXAMPLE 1. Response Must Follow Request Within 5 Cycles

ack must occur between 1 and 5 cycles after req is asserted

```
property p_response;  
  @(posedge clk) req |-> ##[1:5] ack;  
endproperty
```

EXAMPLE 2. Signal Stable for 3 Cycles

data must equal expected_value for 3 consecutive cycles

```
sequence s_stable;  
  data == expected_value [*3];  
endsequence
```

EXAMPLE 3. Pipelined protocol

done must be high exactly 2 cycles after start

```
property p_pipeline;  
  @(posedge clk) start |-> ##2 done;  
endproperty
```

EXAMPLE 4. Data Sent, Acknowledged After N

After send, ack must occur 3 times, spaced arbitrarily in time

```
property p_non_consec_ack;  
  @(posedge clk) send |-> ack [->3];  
endproperty
```

More Sequence Operators

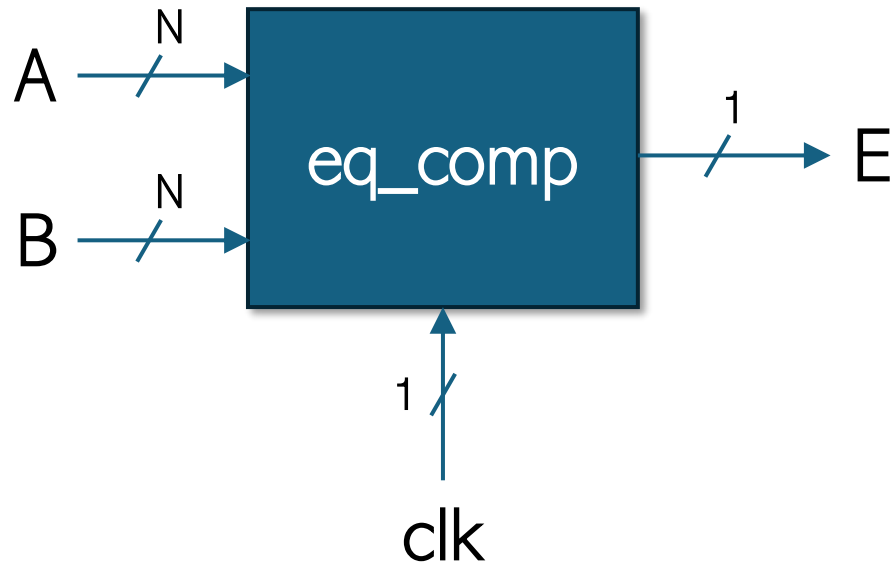
- SV provides powerful temporal operators to build expressive sequences
 - These go beyond `##`, `|->`, and `[*]` to model complex scenarios

Operator	Meaning
within	"Happens during" another sequence <code>a within s</code> True if <code>a</code> occurs entirely within the time window of sequence <code>s</code>
throughout	"Holds for the entire duration" of another sequence <code>a throughout s</code> True if <code>a</code> is continuously true during the whole time <code>s</code> holds
intersect	"Overlap in time" — both sequences occur and share at least one cycle <code>a intersect s</code> True if sequences <code>a</code> and <code>s</code> happen and share at least one common cycle
first_match	Evaluates only the first match of a sequence <code>first_match(s)</code> Returns the first successful match of sequence <code>s</code>

Agenda

1. Introduction to assertions
2. Immediate assertions
3. Concurrent assertions
4. Advanced concurrent assertions
5. A tricky example
6. Wrap-up and self-learning

Example: Syncr N-bit Equal Comparator



```
module eq_comp #(parameter int N = 64) (  
    input  logic      clk,  
    input  logic [N-1:0] A,  
    input  logic [N-1:0] B,  
    output logic      E  
);  
    ...  
endmodule
```

- As `clk` posedge:
 - If `A` is equal to `B`, then `E` becomes true

Describe Behavior Using Properties

FUNCTIONALITY (COMPLETENESS)

```
assert property(  
    @(posedge clk)  
    (A == B) |-> E);
```

- It triggers when $A = B$: then it checks E is true
- It will not fail if E is true when $A \neq B$

CAUSE-EFFECT (SOUNDNESS)

```
assert property(  
    @(posedge clk)  
    E |-> (A == B));
```

- It triggers when E is true: then it checks $A = B$
- It will not fail if $A = B$ and E stays false

You actually need both directions (or an if-and-only-if relationship)

Describe Behavior Using Properties

OPTION #1: DUAL IMPLICATION

```
assert property(
    @(posedge clk)
    (A == B) |-> E);
assert property(
    @(posedge clk)
    E |-> (A == B));
```

OPTION #2: BOOLEAN EQUALITY

```
assert property(
    @(posedge clk)
    E == (A == B));
```

OPTION #3: PROPERTY IFF

```
assert property(
    @(posedge clk)
    E iff (A == B));
```

2-cycle
latency

```
assert property(
    @(posedge clk)
    (A == B) |-> ##2 E);
assert property(
    @(posedge clk)
    E |-> $past(A == B, 2));
```

Most explicit
Excellent for debugging

```
assert property(
    @(posedge clk)
    E == $past(A == B, 2));
```

Industry standard

```
assert property(
    @(posedge clk)
    E iff $past(A == B, 2));
```

Same as previous,
but technically “overkill”
(can be slightly slower)

Agenda

1. Introduction to assertions
2. Immediate assertions
3. Concurrent assertions
4. Advanced concurrent assertions
5. A tricky example
6. Wrap-up and self-learning

Summary of Key Concepts from this Session

■ Assertions – Key Takeaways

- Immediate Assertions: For procedural checks
- Concurrent Assertions: For temporal behaviors
- Assertion Scoping & Binding

■ Big Picture

- Assertions = "Is the design behaving correctly?"

Link to Next Week

- **Week 07: Functional Coverage and Metrics**
 - Understand functional coverage and its importance in verification
 - Learn how to create/integrate covergroups and coverpoints
 - Analyze coverage results and use them to improve verification quality

Self-Learning Corner

■ Books

- SystemVerilog for Verification (3rd Ed.) by Chris Spear
 - Chapter 11: Assertions

■ Videos

- Efficient SystemVerilog Assertions (YouTube link [here](#))

■ Articles

- SystemVerilog Assertions: Design Tricks and SVA Bind Files (link [here](#))

“Assertions: the designer’s way of yelling at future mistakes.”

UVM Mentor quote